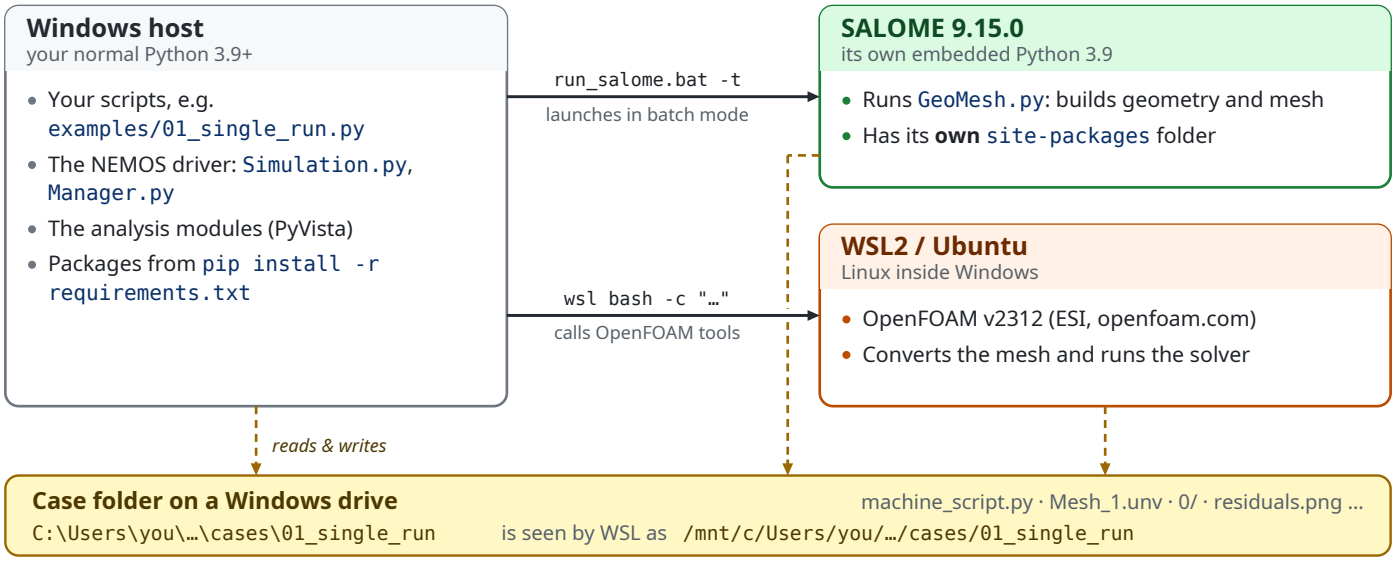
Getting started with NEMOS CFD

Install the toolchain, set up NEMOS and run your first simulation. Follow the seven steps **in order** — each one depends on the one before it.

1. Three environments, one pipeline

NEMOS drives three separate programs from your Windows Python. Every hand-over between them goes through a file in the case folder.



> ⚠ **The three do not share packages.** Installing something with `pip` on Windows does **not** make it available inside SALOME, and OpenFOAM lives only inside WSL. That is why step 6 copies a file into SALOME by hand.

2. The seven steps

1	**Windows and Python** Check Python 3.9+ is on PATH.	PowerShell	page 2
2	**WSL2 with Ubuntu** Install the Linux layer that will run OpenFOAM.	PowerShell (Admin)	page 2
3	**OpenFOAM v2312** Install the ESI release inside Ubuntu, then verify from Windows.	Ubuntu + PowerShell	page 2
4	**SALOME 9.15.0** Install to C:\SALOME-9.15.0 — the path NEMOS expects.	Installer	page 3
5	**Get NEMOS** Clone the repository and pip-install the Python packages.	PowerShell	page 3
6	**GeoMesh.py into SALOME** don't skip Copy one file into SALOME's own Python. Most-skipped step!	PowerShell	page 3
7	**Run your first case** Run example 01 and check the results folder.	PowerShell	page 4

Before you begin, have ready

- [ ] A Windows 10 or 11 PC
- [ ] Administrator rights (to install WSL)
- [ ] An internet connection
- [ ] Git for Windows — or download the ZIP from GitHub
- [ ] 4 free CPU cores (example 01 solves in parallel)
- [ ] 15–40 minutes for the first run

Colour key used in every NEMOS diagram

- Windows host — Simulation.py, Manager.py
- SALOME, running GeoMesh.py
- WSL2 / OpenFOAM
- Your own scripts
- Files in the case folder
- Post-processing (PyVista)

Part 1 — Windows, WSL2 and OpenFOAM

Type commands from the grey boxes into the window named above each box. Text after **#** is a comment — don't type it.

1 Windows and Python

Windows host

You need **Windows 10 or 11** and **Python 3.9 or newer** on your `PATH`. Check it:

PowerShell

```
python --version
```

Expect: `Python 3.9.x` or any higher version is printed.

> **Command not found?** Install Python from python.org and tick **"Add python.exe to PATH"** in the installer, then open a new PowerShell window.

Why Windows? NEMOS translates your case paths (`C:\Users\...`) into WSL paths (`/mnt/c/Users/...`) every time it calls OpenFOAM.

2 WSL2 with Ubuntu

WSL / OpenFOAM

WSL2 runs a real Ubuntu Linux inside Windows; OpenFOAM will live there. Right-click PowerShell → **Run as administrator**, then:

PowerShell (as Administrator)

```
wsl --install -d Ubuntu-24.04
wsl --status   # confirm "Default Version: 2"
```

Expect: `wsl --status` shows **Default Version: 2**. (Ubuntu 22.04 works as well.)

> **First time only:** Windows may ask you to restart. When Ubuntu opens for the first time it asks you to create a Linux username and password — remember the password, `sudo` in step 3 asks for it.

3 OpenFOAM v2312 inside WSL

WSL / OpenFOAM

> **Install the ESI release (openfoam.com), not** the Foundation `openfoamN` packages — their paths and solver set differ, and NEMOS will not find them.

Type `wsl` in PowerShell to enter Ubuntu, then install:

Ubuntu (WSL) terminal

```
wsl   # enters Ubuntu; the next lines run inside it
sudo sh -c "wget -O - https://dl.openfoam.com/add-debian-repo.sh | bash"
sudo apt-get update
sudo apt-get install openfoam2312-default
```

Type `exit` to return to PowerShell and verify **from the Windows side**, exactly as NEMOS will call it:

PowerShell

```
wsl bash -c "source /usr/lib/openfoam/openfoam2312/etc/bashrc && which buoyantSimpleFoam
↪  ideasUnvToFoam decomposePar"
```

↪ *marks a line wrapped to fit the page — type it as one single line.*

Expect: three paths are printed, one per tool.

If not: If the `source` line fails, list the real folder with `ls /usr/lib/openfoam/` (inside Ubuntu) and update the path in `src/Simulation.py` — lines **1227** and **1298**.

Part 2 — SALOME and NEMOS itself

SALOME builds the geometry and the mesh. NEMOS drives it in batch mode, so you never have to open its window — though the GUI is handy for inspecting a mesh.

4 SALOME 9.15.0

SALOME

Download the **Windows binary** from `salome-platform.org` and install it. Keep the default location `C:\SALOME-9.15.0` — that is where NEMOS looks. Confirm the launcher exists:

PowerShell

```
Test-Path "C:\SALOME-9.15.0\run_salome.bat"
```

Expect: `True`.

> **Installed somewhere else?** Either edit `DEFAULT_SALOME_EXE` in `src/Simulation.py` (line 13), or pass the path on every run: `nemos.run(sim, salome_exe=r"D:\SALOME-9.15.0\run_salome.bat")`

5 Get NEMOS and its Python packages

Windows host

Pick a folder on a Windows drive (for example `C:\Users\you\`), then clone the repository and install the host-side packages:

PowerShell

```
git clone https://github.com/pedro-oliveira7r/NEMOS_CFD.git
cd NEMOS_CFD
pip install -r requirements.txt
```

Expect: pip finishes without errors. It installs matplotlib, numpy, pandas, pyvista, scipy and openpyxl — into your **Windows** Python only.

> **No Git?** On the GitHub page choose **Code → Download ZIP**, extract it, and `cd` into the extracted folder instead.

6 Install GeoMesh.py into SALOME

most likely to be skipped SALOME

`src/GeoMesh.py` runs **inside SALOME**, and SALOME cannot see your clone. The script NEMOS generates (`machine_script.py`) starts with `import GeoMesh as nemos`, so the module must sit in SALOME's own `site-packages`.

Your clone (Windows Python)
`NEMOS_CFD\src\GeoMesh.py`
SALOME cannot see this folder.
This is the copy you edit.

Copy-Item → by hand, after every edit

SALOME's own Python
`C:\SALOME-9.15.0\W64\Python\lib\site-packages\GeoMesh.py`
This is the copy SALOME actually runs.

PowerShell · run inside the NEMOS_CFD folder

```
Copy-Item src\GeoMesh.py "C:\SALOME-9.15.0\W64\Python\lib\site-packages\GeoMesh.py"
Test-Path "C:\SALOME-9.15.0\W64\Python\lib\site-packages\GeoMesh.py"
```

Expect: `Test-Path` prints `True`.

> **Re-copy after every change to GeoMesh.py.** There is no link between the two copies — a stale copy silently builds the **old** geometry.

> **Nothing else to install in SALOME.** SALOME 9.15.0 already ships pandas 1.5.3 and numpy 1.21.6 — all that `GeoMesh.py` imports.

Part 3 — Run your first case

Example 01 is the smallest complete NEMOS pipeline and the only cheap example — use it to prove that steps 1–6 worked before trying anything bigger.

7 Run the smoke test: example 01

Your script

PowerShell · run inside the NEMOS_CFD folder

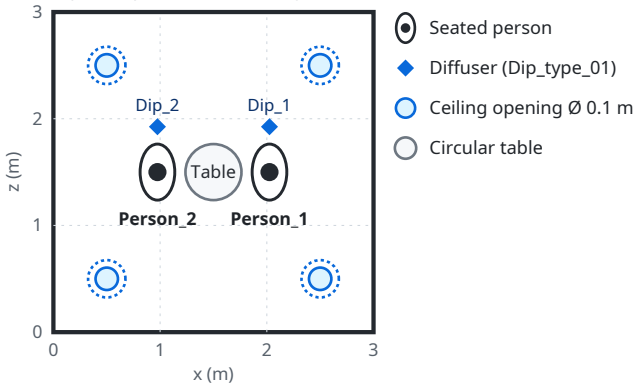
```
python examples/01_single_run.py
```

Time: roughly **15–40 minutes** — meshing takes most of it.

Also: a residuals plot window opens while it solves. That is normal.

What you are simulating — a 3 × 3 × 3 m room with two seated people at a round table, a personal air diffuser at each person, and four openings in the ceiling:

Top view · positions to scale, object sizes schematic

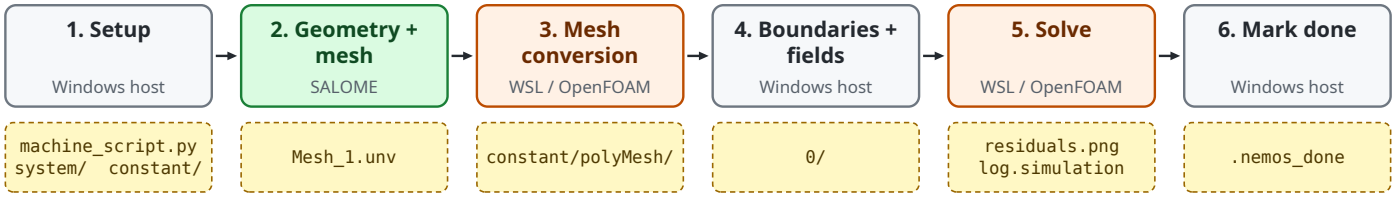


Object	Position (m)	Value
Room	3 × 3 × 3	walls 300 K
Person_1, seated	x 2.025 · z 1.5	304 K
Person_2, seated	x 0.975 · z 1.5	305 K
Dip_1, diffuser	x 2.025 · y 1.0 · z 1.5	air 291 K
Dip_2, diffuser	x 0.975 · y 1.2 · z 1.5	air 293 K
Table, circular	x 1.5 · y 0.6 · z 1.5	300 K
4 ceiling openings	the four corners	Ø 0.1 m

y is height above the floor. Each boundary value is written next to its object in the script (`T_Face`, `T_Flow`, `V_Flow`) and matched to the mesh by its `name=`.

What happens when you press Enter

Your script only **records** what you describe; nothing is built until the last line, `nemos.run(sim)`. Then six stages run in order, each one leaving a file for the next:



Safe to re-run. Each stage is **skipped if its output already exists**, and the solver continues from its last saved step. If the run is interrupted, just start the same command again.

Did it work?

It succeeded if the case folder contains the four ticked items:

```
cases/01_single_run/
✓ ├─ Mesh_1.unv              the mesh built by SALOME
✓ ├─ 0/                      all 8 field files: U, T, p, k, …
✓ ├─ residuals.png           convergence plot of the solver
✓ ├─ .nemos_done             written last — the run finished
  ├─ constant/  system/      OpenFOAM case set-up
  ├─ processor0/ … processor3/   the 4 parallel pieces
  ├─ log.simulation          full solver log
  ├─ machine_script.py       the script SALOME ran
  └─ caso.foam               opened by the analysis tools
```

Changed the geometry? Nothing changes until you delete `Mesh_1.unv` and `.nemos_done` in the case folder — otherwise the old mesh is reused.

Start completely fresh: delete `.nemos_done`, plus `Mesh_1.unv`, `0/` and `processor*/` to rebuild every stage.

Case location: change `CASE_PATH` at the top of the example. It must stay on a Windows drive (C:, D:, …).

When something goes wrong

Find the message you see in the left column. The coloured bar shows which part of the toolchain is involved; the step number points back to where it was set up.

You see	Why	Fix	Step
`ModuleNotFoundError: No module named 'GeoMesh'` **in the SALOME output**	Step 6 was skipped, or SALOME is installed somewhere else	Copy `GeoMesh.py` into SALOME's `site-packages`	6
`ERRO: Executável do SALOME não encontrado em: ...`	Wrong `DEFAULT_SALOME_EXE`	Fix `src/Simulation.py` line 13, or pass `salome_exe=`	4
`ideasUnvToFoam: command not found`	The OpenFOAM `bashrc` did not source	Compare the path in `Simulation.py` lines 1227 / 1298 with `ls /usr/lib/openfoam/`	3
`wsl: command not found` **/ WSL errors**	WSL2 not installed, or no default distro	`wsl --install -d Ubuntu-24.04`, then `wsl --set-default Ubuntu-24.04`	2
Solver hangs immediately	`mpirun` cannot get 4 slots	Use `parallel="N"`, or free up cores — the 4 subdomains are hardcoded	7
Geometry changes have no effect	A stale `Mesh_1.unv` is being reused	Delete `Mesh_1.unv` and `.nemos_done` in the case folder	7
SALOME finishes but no `Mesh_1.unv` **appears**	`create_mesh(export=True)` was omitted	`export=True` is what injects the export path	—
The run seems to block on a plot	`plot_residuals=True` drains the output synchronously	Expected. Pass `plot_residuals=False` for an async run	7
`Todas as listas devem ter o mesmo tamanho`	Sweep lists of differing length	All top-level lists in a Manager script must be the same non-zero length	—
`FileNotFoundError: .../caso.foam` **during analysis**	Case built by an older NEMOS version	Create an empty `caso.foam` at the case root (example 07 does this for you)	—
Analysis sees only time 0	Parallel run was never reconstructed	Run `reconstructPar` in the case (command below)	—

PowerShell · rebuilds the full result from the processor* folders

```
wsl bash -c "source /usr/lib/openfoam/openfoam2312/etc/bashrc && cd /mnt/c/<path to your case> &&
↪ reconstructPar"
```

↪ *marks a line wrapped to fit the page — type it as one single line.*

Where your cases may live

Case paths must be on a Windows drive that WSL can reach through `/mnt/`. Anything else is not translated and fails.

✓ `C:\Users\you\cases\run1`	local drive
✓ `D:\cfd\run1`	second local drive
✗ `\\server\share\run1`	network share
✗ `Z:\run1`	mapped drive
✗ `\\wsl$\Ubuntu\home\…`	inside WSL's filesystem

Where to go next

✓ **Install the toolchain**	this guide, steps 1–6
✓ **Run example 01**	step 7 — you are here
3 **Describe your own room**	`docs/api-reference.md` — every class and keyword argument
4 **Look at the results**	`examples/06_analysis.py` — run `reconstructPar` first if the case ran in parallel
5 **Run a parametric study**	`examples/03_sweep_conditions.py` — five full runs back to back; takes hours

The one idea to hold on to: NEMOS is split across two Python interpreters. The classes you call in your script (`Room`, `Person_seated`, `Mesh`) only **record** a script; SALOME replays it later against the same-named classes in `GeoMesh.py`. That is why every object needs a `name=`, and why `GeoMesh.py` must be copied into SALOME.